Playwright AI Automation with MCP, Prompts, Instructions, Context, Skills, Agents, and CLI

Complete practical guide for VS Code GitHub Copilot and JavaScript Playwright

Target project: OrangeHRM login automation\ **Language:** JavaScript\ **Test runner:** `@playwright/test`\ **Design pattern:** Page Object Model (POM)\ **Application:** `https://opensource-demo.orangehrmlive.com/web/index.php/auth/login`

This guide explains what MCP, prompts, instructions, context, skills, agents, and CLI mean, how they differ, how to configure them in VS Code, and the recommended order to use them for a reliable Playwright automation workflow.

Compatibility note: *VS Code Copilot Chat, Copilot CLI, and other agent hosts do not have identical customization formats, commands, or discovery rules. This guide focuses primarily on current VS Code Copilot agent customization, with CLI notes where relevant. Always check the documentation for the exact host you are using.*

1. The big picture

Think of an AI automation setup as a small team:

- **MCP** connects the AI to external tools and services.
- **Tools** are actions the AI can perform, such as reading files, running commands, or controlling a browser.
- **Instructions** define rules the AI should follow.
- **Context** supplies project facts and requirements.
- **Skills** describe reusable procedures for doing work.
- **Prompts** are reusable task requests.
- **Agents** define specialized roles, instructions, and tool access.
- **CLI** is a command-line interface used to run commands and interact with development tools, including agent CLIs.

```
User request
    |
    v
Select the appropriate agent
    |
    +--> Read applicable instructions
    +--> Load relevant context
    +--> Invoke a prompt or skill
    +--> Use available tools
              |
              +--> Workspace/file tools
              +--> Terminal tools
              +--> Browser/Playwright tools
```

```
              +--> MCP-provided external tools
    |
    v
Inspect evidence and validate
    |
    v
Report changes and real test results
```

The host controls the exact loading order and orchestration. This diagram is a mental model, not a guarantee that every file is automatically loaded.

2. Quick comparison

---------- Concept Main question it Playwright example Typical artifact answers ---------- ---------- ---------- ---------- MCP How can the agent A Playwright MCP server exposes `.mcp.json` or host-specific MCP connect to browser actions. configuration external tools/services?

Tool What action can Read a page, click a button, Built-in tool, extension tool, or MCP the agent perform? run a terminal command. tool

Instructions What rules should Use JavaScript, POM, web-first `.github/copilot-instructions.md` or always apply? assertions, and safe credential `*.instructions.md` handling.

Context What App URLs, existing `context/*.md`, requirements, specs, project-specific architecture, test IDs, source files information environment setup. matters?

Skill How should a Analyze Excel tests or heal `.github/skills/<skill-name>/SKILL.md` category of work failing tests using evidence. be done?

Prompt What reusable task Generate tests from the `.github/prompts/*.prompt.md` should be approved plan. requested?

Agent Which role `orangehrm-playwright-agent`. `.github/agents/*.agent.md` coordinates this kind of task?

CLI How do I run tools `npx playwright test`, Commands, scripts, terminal sessions or commands from a `npx playwright show-report`, terminal? or `copilot`.

Important: CLI is not another type of instruction file. It is a way to invoke a program from the command line.

3. MCP --- Model Context Protocol

What is MCP?

MCP stands for **Model Context Protocol**. It is an open standard for connecting AI applications to external tools and services. An MCP server can expose tools and, depending on the server, resources, prompts, or interactive apps.

An MCP server might let an agent interact with a browser, query a database, retrieve issue information, access approved documentation, or invoke an external service.

MCP is a connection mechanism. It is not the AI model, not the agent's role, and not a Playwright test suite.

Do you need Playwright MCP?

Not always. VS Code may already have browser tools, and other tools may cover the task. Use MCP when you need a capability that is not already available.

A Playwright MCP server can expose browser actions for navigating pages, inspecting content, clicking, filling forms, and taking screenshots. Exact tools depend on the server.

Interactive browser automation via MCP is not the same as a committed regression suite. Use `@playwright/test` files for repeatable local and CI tests.

Add an MCP server in VS Code

1. Open the Extensions view.
2. Search for `@mcp playwright` to discover a Playwright MCP server.
3. Inspect its publisher, source, capabilities, and configuration.
4. Install only if appropriate for your environment.
5. Review and trust the server configuration before starting it.
6. Open Copilot Chat and inspect the available tools.
7. Try a low-risk browser task and verify the actual result.

Current VS Code configurations support a workspace-root `.mcp.json` destination and user-level configuration. Older paths may appear for compatibility; use the destination offered by your installed UI and current documentation.

Do not copy an unverified MCP configuration from the internet. A local MCP server may run code on your machine, and a server may access data or external systems. Inspect its command, arguments, environment variables, permissions, and source before trusting it.

Example MCP browser request

```
Use the available Playwright browser tools to open the OrangeHRM login page.
Inspect the visible username field, password field, Login button, and heading.
Report their accessible roles and names. Do not enter credentials or modify files.
```

If the tool is unavailable, ask the agent to say so. Installing Playwright in `node_modules` does not automatically install or expose an MCP browser server.

MCP safety

- Install only servers you trust.
- Inspect workspace MCP configuration before trusting a repository.
- Avoid putting secrets in configuration files.
- Use least-privilege credentials.
- Do not permit destructive external actions without explicit

approval.

- Keep exploration and test accounts separate from real customer data

where possible.

4. Instructions --- standing rules

Instructions are persistent guidance for the agent. They specify conventions and constraints that should apply across relevant tasks.

Use instructions for stable rules such as language/framework, architecture, coding style, security requirements, validation expectations, and things the agent must not change.

Common locations

Repository-wide GitHub Copilot instructions are commonly stored in:

```
.github/copilot-instructions.md
```

VS Code also supports scoped instruction files, commonly named:

```
.github/instructions/playwright.instructions.md
```

The exact discovery and metadata rules depend on the host. Use the Agent Customizations editor if you are unsure of the format.

Example instructions

```
# OrangeHRM Playwright Project Rules

- Use JavaScript and `@playwright/test`.
- Follow the existing Page Object Model conventions.
- Inspect the repository before editing.
- Read `context/context.md` and task-specific context as needed.
- Read the relevant `SKILL.md` before specialized workflows.
- Preserve real test IDs, steps, and expected results from the manual workbook.
- Prefer accessible locators and Playwright web-first assertions.
- Avoid arbitrary fixed sleeps.
- Reuse existing fixtures, helpers, and configuration.
- Keep credentials in environment variables or approved secret storage.
- Never print passwords or commit secrets.
- Never weaken assertions or silently skip failures to force a green run.
- Never claim tests passed unless actually executed successfully.
- Report commands executed, files changed, and actual results.
```

Do not put every possible procedure into the instruction file. Keep stable rules in instructions, facts in context, detailed procedures in skills, and reusable requests in prompts.

5. Context --- project knowledge and evidence

Context is information needed to perform a task correctly: project architecture, application URLs, workflows, existing code, manual tests, configuration, environment restrictions, prior decisions, logs, screenshots, and traces.

Context is not necessarily a special Copilot file type. It can be ordinary Markdown, code, JSON, a specification, a workbook, or another accessible artifact. The agent can use only information it can actually access.

Suggested context layout

```
context/
■■■ context.md
■■■ 01_testcase_analysis.md
■■■ 02_pom_generation.md
■■■ 03_test_generation.md
■■■ 04_framework_integration.md
■■■ 05_test_healing.md
■■■ 06_agent_workflow.md
```

---------- File Contents ----------
---------- `context.md` Project overview, application, framework, directory structure, rules, environment

`01_testcase_analysis.md` How to inspect the real manual workbook and interpret test rows

`02_pom_generation.md` POM design rules and existing framework patterns

`03_test_generation.md` Test naming, assertions, data, fixtures, and file layout

`04_framework_integration.md` Configuration, utilities, reporting, CI and commands

`05_test_healing.md` Diagnosis using evidence and minimal fixes

`06_agent_workflow.md` End-to-end process, checkpoints, approvals, deliverables

These are proposed names. If your repository already uses different filenames, keep its structure or update references consistently.

Example `context/context.md`

```
# Project Context

## Application
- Application: OrangeHRM demo
- Login URL: https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
- Successful dashboard URL pattern: /web/index.php/dashboard/index

## Technology
- Language: JavaScript
- Test runner: @playwright/test
- Pattern: Page Object Model

## Workflow
1. Inspect the actual manual workbook.
2. Prepare a test plan.
3. Review and approve the plan.
4. Generate or update POMs and tests.
5. Run tests and inspect actual results.
6. Diagnose failures using evidence.
7. Apply minimal fixes and rerun.

## Constraints
- Reuse existing project code.
- Do not invent test cases or expected results.
- Do not hardcode credentials.
- Do not claim an unexecuted test passed.
```

Never put live passwords, API keys, or private customer data in context files. Use approved secret storage or environment variables.

How to submit context to an agent

Do not assume every file is automatically loaded. Explicitly ask the agent to: 1. Read the master context file. 2. Identify the task type. 3. Discover and read relevant task-specific context. 4. Inspect the actual workbook, code, or evidence. 5. Report missing files or ambiguous requirements.

Example:

```
Read `.github/copilot-instructions.md` and `context/context.md`.
For test analysis, also read `context/01_testcase_analysis.md`.
Locate and inspect the actual workbook. List the context files you read
and report missing files or ambiguous requirements before planning.
```

Use workspace file references or the host's supported file-reference syntax when available.

6. Skills --- reusable procedures

A skill packages a repeatable procedure for a category of work. In VS Code, an agent skill commonly consists of a `SKILL.md` file with YAML metadata and instructions. It can include supporting files, templates, or scripts.

Suggested Playwright skills

```
.github/skills/
■■■ excel-test-analysis/SKILL.md
■■■ pom-design/SKILL.md
■■■ javascript-playwright-tests/SKILL.md
■■■ framework-integration/SKILL.md
■■■ test-healing/SKILL.md
■■■ orangehrm-agent-workflow/SKILL.md
```

In current VS Code, workspace skills are commonly stored under `.github/skills/<skill-name>/SKILL.md`. The host determines exact discovery rules.

Example skill

File: `.github/skills/javascript-playwright-tests/SKILL.md`

```
---
name: javascript-playwright-tests
description: Generate and validate JavaScript Playwright Test cases from an approved test plan using
 the existing POM framework.
---

# JavaScript Playwright Test Generation

## When to use
Use when the user asks to generate or update Playwright tests.

## Procedure
1. Read project instructions and master context.
2. Read `context/03_test_generation.md`.
3. Read the approved `artifacts/test-plan.md`.
4. Inspect current config, tests, POMs, fixtures, and utilities.
5. Reuse project structure and conventions.
6. Generate JavaScript tests using `@playwright/test`.
7. Preserve approved test IDs and expected outcomes.
8. Use accessible locators and web-first assertions.
9. Avoid fixed sleeps and hardcoded credentials.
10. Run test discovery and relevant tests when permitted.
11. Report changed files and actual test results.

## Guardrails
```

```
- Do not invent test cases.
- Do not weaken assertions to force a pass.
- Do not silently skip failures.
- Do not claim tests passed without execution evidence.
```

Skills may appear as slash commands and may be automatically loaded when relevant, depending on the host. Their descriptions should clearly explain when to use them.

7. Prompts --- reusable task requests

A prompt file is a saved request template for a common task. In VS Code, workspace prompt files commonly use `.prompt.md` under `.github/prompts/`.

Prompts are normally invoked explicitly. They standardize the request and expected output.

Suggested prompt files

```
.github/prompts/
■■■ orangehrm-analyze-workbook.prompt.md
■■■ orangehrm-generate-tests.prompt.md
■■■ orangehrm-run-tests.prompt.md
■■■ orangehrm-heal-failures.prompt.md
```

Example prompt file

File: `.github/prompts/orangehrm-analyze-workbook.prompt.md`

```
---
name: orangehrm-analyze-workbook
description: Analyze the actual OrangeHRM manual test workbook and prepare a reviewable test plan.
agent: agent
---

Read the repository instructions and `context/context.md`.
Read `context/01_testcase_analysis.md` and the relevant Excel analysis skill.

Locate and inspect the actual manual test workbook. Preserve its real
test IDs, steps, and expected results.

Create or update `artifacts/test-plan.md`.

Do not generate or modify Playwright test code yet.
Do not invent test cases if the workbook is missing or ambiguous.
Report files read, workbook location, test cases identified, and questions
that require user approval.
```

Run a prompt by typing `/` and selecting its name, or use the supported prompt-run command. Exact invocation depends on the current VS Code version.

Prompt versus skill

- **Prompt:** the request to perform a task now.
- **Skill:** the reusable procedure for performing that kind of task.

A prompt can reference a skill; a skill can explain how to carry out tasks triggered by different prompts.

8. Agents --- specialized roles

A custom agent defines a reusable role for a category of work. In VS Code, custom agents commonly use `.agent.md` files in `.github/agents/`. The agent definition can include its name, description, available tools, and operating instructions.

Example coordinator agent

File: `.github/agents/orangehrm-playwright-agent.agent.md`

```
---
name: orangehrm-playwright-agent
description: Coordinates OrangeHRM QA automation using JavaScript, Playwright Test, POM, project con
text, skills, and test evidence.
---

# Role

You coordinate the OrangeHRM Playwright automation project.

## Before each task
1. Read `.github/copilot-instructions.md` if present.
2. Read `context/context.md` if present.
3. Identify the requested task: analysis, planning, POM, generation,
   integration, execution, or healing.
4. Discover and read relevant task-specific context and `SKILL.md` files.
5. Inspect existing files before editing.
6. Report missing required inputs instead of guessing.

## Workflow
- Analysis: inspect the actual manual workbook.
- Planning: create `artifacts/test-plan.md`; seek approval if scope is ambiguous.
- Generation: use the approved plan and existing framework conventions.
- Integration: reuse fixtures, utilities, and configuration.
- Execution: run permitted commands and inspect actual results.
- Healing: diagnose with errors, traces, screenshots, and DOM evidence;
  make the smallest justified fix and rerun.
- Reporting: list files changed, commands executed, actual test outcomes,
  unresolved issues, and next steps.

## Rules
- Use JavaScript and `@playwright/test`.
- Follow existing POM conventions.
- Never invent workbook rows or expected results.
- Never hardcode credentials.
- Never weaken assertions or hide failures to force a pass.
- Never claim a test passed without execution evidence.
- Ask before destructive or broad changes.
```

The exact frontmatter and tool selection depend on the active host. Create the initial file through the Agent Customizations UI if you want compatible metadata.

One agent or many?

Start with **one coordinator agent**. Add specialist agents only when the work justifies it: - Test Plan Analyst --- read-only workbook analysis. - Playwright Test Generator --- generates code from an approved plan. - Test Failure Analyst --- investigates failures and proposes minimal fixes. - Code Reviewer --- reviews diffs and test quality.

Give each agent a clear boundary. Avoid multiple agents editing the same files simultaneously unless you have conflict management.

9. CLI --- command-line interface

CLI means **Command-Line Interface**. It is not a prompt, skill, or agent file. It lets you run commands from a terminal.

For a Playwright project, the Playwright CLI runs tests and related tasks. GitHub Copilot CLI, if installed and configured, provides a command-line interface for an AI coding agent. VS Code's terminal can run both, but they are different programs.

Common Playwright commands

Run these from the project root.

Install Playwright Test if it is not installed:

```
npm install -D @playwright/test
npx playwright install
```

List tests without executing:

```
npx playwright test --list
```

Run the full suite:

```
npx playwright test
```

Run one file:

```
npx playwright test tests/login.spec.js
```

Run tests matching a title:

```
npx playwright test -g "valid login"
```

Open the HTML report:

```
npx playwright show-report
```

Show Playwright CLI help:

```
npx playwright --help
```

If Copilot CLI is installed:

```
copilot --help
```

Do not assume VS Code's `.prompt.md`, `.agent.md`, MCP, and skill configuration behaves identically in every CLI version. Follow the active host's official documentation.

How CLI fits into automation

```
Agent proposes a code change
       |
```

```
       v
Terminal tool runs Playwright CLI
       |
       v
Actual process output and reports
       |
       v
Agent diagnoses evidence
       |
       v
Minimal fix, then rerun
```

The agent must distinguish commands it recommended from commands it actually executed.

10. Tools --- the actions agents can invoke

A tool is a capability exposed to the agent. Tools may be built into VS Code, contributed by extensions, or provided by MCP servers.

Examples include reading or editing a file, searching workspace text, running a terminal command, interacting with a browser, or retrieving information from an external service.

A prompt or skill cannot magically provide a tool that is not available. If the agent needs browser interaction, ensure a suitable browser tool or Playwright MCP server is enabled. If it needs to run tests, ensure terminal execution is available and permitted.

---------- Browser exploration tool Playwright Test suite
---------- ---------- Good for interactive inspection Good for repeatable regression tests

Useful for discovering accessible Useful for assertions in code names

May be session-specific Can run locally and in CI

Does not automatically create a Test lives in the repository durable test

Availability depends on host Depends on project dependencies and environment
---

Use browser tools to investigate. Use Playwright tests to preserve and repeat verified behavior.

11. Correct end-to-end OrangeHRM workflow

Use stages and review checkpoints rather than asking the agent to do everything in one large prompt.

Stage 0 --- prepare the repository

Confirm the workspace root is the actual Playwright project. Inspect `package.json`, `playwright.config.js`, tests, pages, fixtures, and utilities.

Example layout:

```
PlaywrightTest/
```

```
■■■ .github/
■   ■■■ copilot-instructions.md
■   ■■■ agents/orangehrm-playwright-agent.agent.md
■   ■■■ instructions/playwright.instructions.md
■   ■■■ prompts/
■   ■   ■■■ orangehrm-analyze-workbook.prompt.md
■   ■   ■■■ orangehrm-generate-tests.prompt.md
■   ■   ■■■ orangehrm-heal-failures.prompt.md
■   ■■■ skills/
■       ■■■ excel-test-analysis/SKILL.md
■       ■■■ pom-design/SKILL.md
■       ■■■ javascript-playwright-tests/SKILL.md
■       ■■■ framework-integration/SKILL.md
■       ■■■ test-healing/SKILL.md
■■■ context/
■   ■■■ context.md
■   ■■■ 01_testcase_analysis.md
■   ■■■ 02_pom_generation.md
■   ■■■ 03_test_generation.md
■   ■■■ 04_framework_integration.md
■   ■■■ 05_test_healing.md
■   ■■■ 06_agent_workflow.md
■■■ ManualTests/testcase.xlsx
■■■ pages/
■■■ tests/
■■■ artifacts/test-plan.md
■■■ playwright.config.js
■■■ package.json
```

This is an example layout. Preserve the real workbook name and existing project structure.

Stage 1 --- inspect only

Select `orangehrm-playwright-agent` in VS Code and submit:

```
Inspect this repository without editing files.

Read the project instructions and master context. Discover the actual
context files, skills, prompts, and existing test framework files.
Locate the real manual workbook.

Report:
1. Which files you read.
2. Existing POMs, tests, fixtures, utilities, and configuration.
3. Required files that are missing.
4. Blockers or ambiguities.

Do not create or modify code yet.
```

Checkpoint: Review the report before continuing.

Stage 2 --- analyze workbook and prepare plan

```
Use the excel-test-analysis skill if available.
Read context/01_testcase_analysis.md.
Inspect the actual manual workbook.

Create artifacts/test-plan.md using only the real workbook contents.
Preserve test IDs, steps, expected results, preconditions, and dependencies.
Do not generate Playwright code yet. Flag ambiguous rows instead of guessing.
```

Checkpoint: Review and approve the plan.

Stage 3 --- design or reuse POMs

```
Read context/02_pom_generation.md and the pom-design skill if present.
Inspect existing pages, tests, fixtures, and utilities.
```

```
Recommend which POMs should be reused, updated, or added. Do not duplicate
existing abstractions. Explain proposed changes before broad edits.
```

Stage 4 --- generate tests

```
Use the approved artifacts/test-plan.md.
Read context/03_test_generation.md and relevant JavaScript Playwright/POM skills.

Generate JavaScript @playwright/test files following existing conventions.
Preserve approved test IDs and expected outcomes. Use reliable locators,
web-first assertions, and environment variables for credentials.
Do not invent test cases or weaken assertions.
```

Stage 5 --- integrate and validate

```
Read context/04_framework_integration.md and the framework-integration
skill if present. Inspect actual configuration and package scripts.
Integrate tests using existing fixtures, utilities, and reporting.
Run test discovery and relevant tests if permitted. Report exact commands
and actual outcomes.
```

Typical PowerShell commands:

```
$env:ORANGEHRM_USERNAME = "Admin"
$env:ORANGEHRM_PASSWORD = "admin123"

npx playwright test --list
npx playwright test
npx playwright show-report
```

These example credentials are commonly used for the public OrangeHRM demo, but their continued validity is not guaranteed. Use only authorized test accounts.

Stage 6 --- heal a failure

```
Read context/05_test_healing.md and the test-healing skill.
Inspect actual failure output and relevant screenshot, trace, DOM, and
console evidence. Classify the root cause before editing.

Make the smallest justified fix. Do not remove assertions, silently skip
tests, or treat an unexecuted test as passing. Rerun the focused test and
relevant regression tests. Record actual results in
artifacts/healing-report.md.
```

Stage 7 --- review

```
Review the current diff for test correctness, locator reliability,
assertion quality, POM consistency, credential safety, and duplication.
Report findings with file references. Do not change files until I approve
the proposed fixes.
```

Stage 8 --- report actual results

Require: - Files read and changed. - Commands actually executed. - Passed, failed, skipped, and not-run counts. - Root cause and evidence for each repair. - Remaining issues. - Recommended next step.

12. How to submit work to an AI agent

Use a consistent task template.

```
ROLE:
Use the orangehrm-playwright-agent.

TASK:
Analyze the actual manual workbook and prepare a test plan.

READ:
- .github/copilot-instructions.md
- context/context.md
- context/01_testcase_analysis.md
- Relevant .github/skills/.../SKILL.md

INPUT:
The actual workbook in ManualTests/.

CONSTRAINTS:
- Do not invent test cases.
- Preserve workbook IDs and expected results.
- Do not edit tests yet.
- Report missing files and ambiguous rows.

OUTPUT:
- artifacts/test-plan.md
- A short summary of files read and blockers.

VALIDATION:
Confirm every planned test maps to a real workbook row.
Do not claim any tests ran.
```

For complex tasks, separate requests into: 1. Inspect. 2. Plan. 3. Approve. 4. Edit. 5. Run. 6. Heal. 7. Review and report.

This is safer than asking "automate everything" in a single prompt.

13. Recommended minimal setup

Start with: 1. `.github/copilot-instructions.md` --- stable rules. 2. `context/context.md` --- project overview. 3. Task-specific context files --- detailed requirements and architecture. 4. `.github/skills/` --- reusable procedures. 5. `.github/agents/orangehrm-playwright-agent.agent.md` --- coordinator. 6. `.github/prompts/` --- repeatable requests. 7. MCP server only if the current tools lack a needed capability.

Do not install MCP just because it is available. First check whether built-in browser and terminal tools already cover the workflow.

14. Official documentation

Use documentation for the exact VS Code/Copilot version in use.

- [VS Code agent customization

overview](https://code.visualstudio.com/docs/agent-customization/overview)

- [Custom agents in VS

Code](https://code.visualstudio.com/docs/agent-customization/custom-agents)

- [Agent skills in VS

Code](https://code.visualstudio.com/docs/agent-customization/agent-skills)

- [Prompt files in VS

Code](https://code.visualstudio.com/docs/agent-customization/prompt-files)

- [MCP servers in VS

Code](https://code.visualstudio.com/docs/agent-customization/mcp-servers)

- Tools with agents
- Playwright Test Agents
- Playwright Test documentation

Final summary

- **Instructions** define the rules.
- **Context** provides the facts.
- **Skills** define the procedures.
- **Prompts** request specific tasks.
- **Agents** coordinate the work.
- **Tools** perform actions.
- **MCP** connects external capabilities.
- **CLI** runs commands and tests.

The recommended flow is:

Inspect → Analyze → Plan → Review → Generate → Integrate → Execute → Heal → Review and report.

Start with instructions, a master context file, one coordinator agent, and a few focused skills. Add prompts to make common tasks repeatable. Add MCP only when you need a capability that is not already available. Always verify generated code and actual test results before accepting the work.